Meridian — Implementation Plan

Date: 2026-10-04 **Spec:** `docs/specs/2026-10-04-meridian-design.md` **Status:** awaiting user review

Note: the brainstorming skill's terminal step calls for a `writing-plans` skill. That skill is not installed and is not published on skills.sh, so this plan was written directly. No other skill was substituted.

Build order and why

The order is driven by one rule: **no module imports a module that does not exist yet.** Each phase ends with runnable code you can exercise before the next phase begins.

```
Phase 1  data/ seed files          no dependencies
Phase 2  store.py + models.py      Phase 1
Phase 3  triage.py                 Phase 1
Phase 4  registry.py               Phase 1, 2 (shares CSV reading)
Phase 5  analytics.py              Phase 1 (datasets already present)
Phase 6  app.py                    Phases 2-5
Phase 7  docs/concepts/*.md        Phase 6
```

Phases 3 and 4 are independent of each other and could run in parallel.

Phase 1 — Seed data files

Why first: every later phase reads these. Getting the real symptom vocabulary in place lets triage be tested against actual content rather than invented strings.

Files to create

File	Rows	Notes
`data/diseases.csv`	15	Knowledge base. Every symptom/medication row must be verifiable — flag any that cannot be and it gets removed, not invented.
`data/medicines.csv`	20	Catalogue, incl. the medications named in `diseases.csv` so cross-lookup has real matches.
`data/stores.csv`	6	Medical stores, each with a pipe-delimited `stock_csv`. Some rows deliberately left blank to exercise the blank-cell rule.
`data/patients.csv`	8	Synthetic patients. Explicitly fictional — no real patient data.
`data/visits.csv`	12	Append-only, several per patient so visit-history retrieval is non-trivial.

Verification

Open each file and confirm: header row matches the spec's schema exactly, every `symptoms` cell is pipe-delimited with no spaces around the pipes, every `stock_csv` contains only ids that exist in `medicines.csv`, and every `patient_id` in `visits.csv` exists in `patients.csv`.

Done when the five files exist, schemas match spec section 4.3, and referential integrity holds.

Phase 2 — `store.py` and `models.py`

Taught concepts: file modes `r/w/a/x`, `with`, `DictReader`/`DictWriter`, `os.listdir`, `os.makedirs`, `os.path.exists`, `os.walk`, classes.

`models.py`

```
class Patient:
    def __init__(self, patient_id, name, age=None, gender=None, phone=None,
                 blood_group=None, area=None, city=None,
                 registered_on=None, notes=""):
        ...

    def to_row(self):        # -> list, field order matching the CSV header
    def to_dict(self):       # -> dict for JSON responses
    @staticmethod
    def from_row(row):       # dict -> Patient

class Visit:
    def __init__(self, visit_id, patient_id, visit_date, symptoms="",
                 diagnosis="", severity="", notes=""):
        ...
```

Unfilled fields default to `""`, never `"N/A"`. This is spec rule 4.2 enforced at the constructor, so no caller can accidentally write a sentinel.

`store.py` — public functions

```
PATIENT_FIELDS  = [...]   # module constants mirroring each CSV header
VISIT_FIELDS    = [...]
MEDICINE_FIELDS = [...]
STORE_FIELDS    = [...]

def ensure_csv(path, fields):        # create with header if absent
def read_all(path):                  # -> (records, skipped)
def append_row(path, fields, row):   # open(path, "a")
def get_by_id(records, id_field, target):   # list comprehension filter
def next_id(records, id_field, prefix):# "P-0001" style
def backup_all():                    # os.walk -> backups/<timestamp>/
```

`read_all` returns `(records, skipped)` rather than just records — spec section 9 requires malformed rows to surface in the API response instead of vanishing.

`next_id` scans existing ids for the max suffix and increments. Zero-padded to 4 digits so ids sort lexicographically, which is why `"P-0007"` rather than `"P-7"`.

Tasks

1. Write `models.py`. Both classes with `to_row`/`to_dict`/`from_row`.
2. Write `store.py` constants and `ensure_csv`.
3. Write `read_all` with the skipped-row list.
4. Write `append_row`, `get_by_id`, `next_id`.
5. Write `backup_all` using `os.walk`.
6. Manual test: append a patient, read it back, run a backup, list the backup dir.

Done when a patient round-trips through the CSV unchanged, `next_id` returns `P-0009` after 8 seeded patients, and `backup_all` writes a timestamped copy of all five CSVs.

Phase 3 — `triage.py`

Taught concepts: set `&`/`-`/union, dicts, `sorted(key=lambda)`, string `.strip()`/`.casefold()`, f-strings, `while` loops.

Pure functions only — no Flask, no file writes beyond reading `diseases.csv`. This makes the whole module testable without a server.

Functions, in order

```
SEVERITY_RANK = {"low":0, "moderate":1, "elevated":2, "high":3, "severe":4}
BAND_WIDTH = 0.05

def normalise_symptoms(symptoms):        # list -> set, stripped + casefolded
def load_diseases(path=...):             # DictReader, adds symptom_set
def score_disease(patient_symptoms, disease):  # the 5-tuple, clamped [0,1]
def apply_severity_bands(results):       # from spec 5.3
def check_contraindications(disease, conditions):  # -> list of warning strings
def parse_age_range(text):               # "35-70" -> (35, 70)
def rank_diseases(symptoms, conditions=None, age=None):  # top 5
def render_report(patient, results, warnings):         # Lab 5 banner format
```

`render_report` must end with the disclaimer `NOT A DIAGNOSIS. TRIAGE GUIDANCE ONLY.` as its final line. This is a hard requirement in spec 5.5, not a nicety.

Tasks

1. `normalise_symptoms` + `load_diseases`. Verify `len(diseases) == 15`.
2. `score_disease` including the empty-input guard and the `[0,1]` clamp.
3. `apply_severity_bands` — the `while` loop with an inner `while`.
4. `check_contraindications` + `parse_age_range`.
5. `rank_diseases` with the `n_match < 2` gate.
6. `render_report` in the `*********` banner format.
7. Test against four real symptom sets: exact cold match, unrelated symptoms, empty list, and a dengue-style multi-symptom case.

Done when the four test cases print expected output, empty input returns zeros without raising, and the report's last line is the disclaimer.

Phase 4 — `registry.py`

Taught concepts: dict `.get()`, `.items()`, nested dicts, list comprehensions, set comprehensions, `sorted(key=lambda)`, `.casefold()`, `.split()`.

FIELD_WEIGHTS per record type

```
FIELD_WEIGHTS = {
    "medicines": {"name":5, "generic":4, "category":3, "form":2,
                  "manufacturer":2, "storage":1},
    "stores":    {"name":5, "type":3, "area":2, "city":2, "hours":1},
    "patients":  {"name":5, "city":2, "area":2, "blood_group":2, "phone":1},
}
```

Kept per-type rather than global, because `category` means something different for a medicine than for a store.

Functions

```
def normalise(text)
def match_score(record, terms, weights)
def search_records(records, query, weights)   # -> list, sorted desc
def stores_with_medicine(name, medicines, stores)
def validate_record(record_type, payload)      # -> (cleaned, errors)
```

`search_records` returns everything scoring above 0 — no cutoff. Spec 6.2: the backend never hides data, the frontend greys weak results.

`validate_record` returns `(cleaned, errors)` rather than raising, so the API can report every problem at

once instead of one per round trip. Lab 1 conditionals: required `name`, numeric `age`/`price`, reject empty strings in required fields.

Tasks

1. `normalise` + `match_score` with the blank-cell skip.
2. `search_records`.
3. `stores_with_medicine`.
4. `validate_record` for all three types.
5. Test the three worked examples from spec 6.1 — "metformin diabetes", a store search, and a partial patient query.

Done when searching "metformin diabetes" returns the two Metformin rows above the glucometer row, and a query with no matches returns `[]` rather than raising.

Phase 5 — `analytics.py`

Taught concepts: `read_csv`, `shape`, `describe`, `isnull`, `select_dtypes`, `groupby`, `value_counts`, `sort_values`, `na_values`, plus `os` and matplotlib.

`fetch_datasets.py`, `stage_datasets.py` and `profile_dataset.py` already exist and are verified. `analytics.py` reads what they produced.

```
MISSING_MARKERS = ["?", "-9", "NA", "N/A", ""]

def read_dataset(name)                # na_values=MISSING_MARKERS
def list_datasets()                   # reads profile_report.csv
def profile(name)                     # describe/groupby/value_counts -> dict
def risk_band(value, thresholds)      # Lab 1 BMI pattern
def risk_bands(name, column, thresholds)
def save_charts(name)                 # matplotlib -> static/charts/
```

All pandas output must be converted to plain Python types before returning — numpy `int64` is not JSON-serialisable and Flask will raise on it. A small `to_native()` helper handles this.

`save_charts` writes explicit `plt.hist(...)` and `plt.bar(...)` calls, not `df.hist()`, so every parameter can be explained in a viva (spec 7.5).

Tasks

1. `read_dataset` + `to_native`.
2. `list_datasets` + `profile` for the three selected datasets.
3. `risk_band` + `risk_bands`.
4. `save_charts` with `plt.savefig`.
5. Test JSON-serialisability of every endpoint payload before moving on.

Done when each of the three datasets profiles without error, every payload passes `json.dumps`, and three PNGs land in `static/charts/`.

Phase 6 — `app.py`

Taught concepts: none new beyond Flask itself, which is the whole point.

Routing only. No business logic — every route calls exactly one function from `store`, `triage`, `registry` or `analytics`.

```
GET    /api/health
```

```
POST   /api/patients
GET    /api/patients
GET    /api/patients/<id>            # patient + their visits joined
POST   /api/visits
GET    /api/medicines
POST   /api/medicines
GET    /api/stores
POST   /api/stores
GET    /api/search/<record_type>?q=  # ?q= required
GET    /api/medicines/<name>/stores
POST   /api/triage
GET    /api/diseases
GET    /api/analytics
GET    /api/analytics/<name>
GET    /api/analytics/<name>/risk-bands
GET    /api/analytics/<name>/charts
POST   /api/backup
```

Every route's error handling comes from spec section 9. The three that must not be get wrong:

- blank `?q=` → 400
- unknown dataset name → 404 listing valid names
- zero search matches → 200 with `{"count":0,"results":[]}`, not 404

Tasks

1. App factory + `/api/health`.
2. Patient and visit routes.
3. Medicine, store and search routes.
4. Triage and diseases routes.
5. Analytics and chart routes.
6. Backup route.
7. Run `app.test_client()` over every route, success and failure.

Done when all 18 routes respond correctly under `test_client()`, and the full demo path works through the dev server in a browser.

Phase 7 — `docs/concepts/`

One file per out-of-syllabus concept (spec 2.2). Each explains: what it is, why the project needed it, a minimal before/after example, and which lab it is closest to.

```
docs/concepts/
├── flask.md
├── csv-module.md
├── pandas-select-dtypes.md
├── pandas-missing-values.md
├── matplotlib-basics.md
├── requests.md
└── json.md
```

Each needs to be explainable to someone who has done Labs 1-6 and nothing more.

Done when seven files exist and every out-of-syllabus import in the codebase is covered by one of them.

Tests

`tests/run_tests.py` — plain functions and `assert`, no pytest (spec 10: pytest is not in any lab, so it would need its own doc).

```
tests/
├── run_tests.py
└── test_triage.py
└── test_store.py
└── test_registry.py
└── test_analytics.py
```

Run with `python tests/run_tests.py`. Prints per-test pass/fail and a total. Exit code 1 on any failure so it can gate a commit later.

Not in this plan

- Frontend (phase 2, separate spec — user will supply UI skills first)
- Authentication, deployment, real patient data — excluded in spec section 11
- Drug-interaction checking beyond single contraindication lookup

---

Open items still unanswered

From spec section 12:

1. Frontend stack — deferred
2. Whether 15 curated diseases is enough for the demo
3. Confirm every `diseases.csv` row is verifiable; unverifiable rows get removed

Items 2 and 3 affect Phase 1 and should be settled before that phase starts.